

Frontend API Integration Summary

Wiring the Next.js frontend to the AgriSmartAPI contract from the Frontend & Claude Integration Bridge Guide.

Prepared for: AgriSmart AI hackathon team **Scope:** frontend/ (Next.js 16 + React 19)

1. New files (API scaffolding)

FILE	PURPOSE
frontend/.env.local	NEXT_PUBLIC_API_BASE_URL=http://localhost:8080
frontend/src/types/api.ts	Full TypeScript data contract: User, Farm, FieldPlot, VisionPrediction, DiagnosisAccordionRecord, Product, FullDiagnosisResponse, Chat types, SustainabilityScore, LiveWeatherResponse, SoilTelemetryResponse, SavedAnswer
frontend/src/lib/api.ts	AgriSmartAPI client — one function per backend endpoint defined in the guide

2. Pages wired to real endpoints

Every wired page keeps its original mock/demo value as a fallback and silently falls back to it if the backend at localhost:8080 is unreachable — the UI never breaks, crashes, or shows a blank screen with the backend offline.

PAGE / COMPONENT	ENDPOINT(S) USED
Dashboard — WeatherCard , SoilMoistureCard , DashboardHeader	getLiveWeather , getSoilTelemetry , getCurrentUser
/weather	getLiveWeather (current conditions block)
/irrigation	getSoilTelemetry (live moisture reading)
/sustainability-score + /improve	getSustainability , updateActionStatus (click a suggestion card to mark it complete)
/scan-leaf flow	Real diagnoseLeaf upload → routed through analyzing → result / treatment-advice , auto-branching to low-confidence when confidence < 60%
/chat-assistant	sendMessage with persistent multi-turn session continuity

<code>/chat-assistant/saved-answers</code>	<code>getSavedAnswers</code> , <code>deleteSavedAnswer</code>
<code>/profile</code>	<code>getCurrentUser</code> , <code>getFarms</code> , <code>getFarmFields</code>
Language switch	Fire-and-forget <code>updateLanguage</code> sync

3. Verification performed

- **PASS** `tsc --noEmit` — no type errors
- **PASS** `eslint` — clean (fixed `set-state-in-effect` and `no-explicit-any` issues surfaced along the way)
- **PASS** `next build` — all 36 routes prerender successfully
- **PASS** Headless-browser pass (Playwright) over dashboard, weather, sustainability, irrigation, scan-leaf, chat-assistant, and profile **with no backend running** — every page renders its fallback demo content correctly, no crashes or blank screens

4. Gaps flagged for the backend team

These were intentionally left unwired because the integration guide's contract has no matching endpoint yet — per the guide's own note, gaps get flagged back rather than faked.

- **/history** and **/history/diagnosis** — no "list past diagnoses" endpoint exists in the contract (only chat sessions do)
- **onboarding/farm-profile** — no create-farm endpoint provided (contract is GET-only for farms/fields)
- **Chat image attachments** — the `/chat` endpoint only accepts text, not file uploads